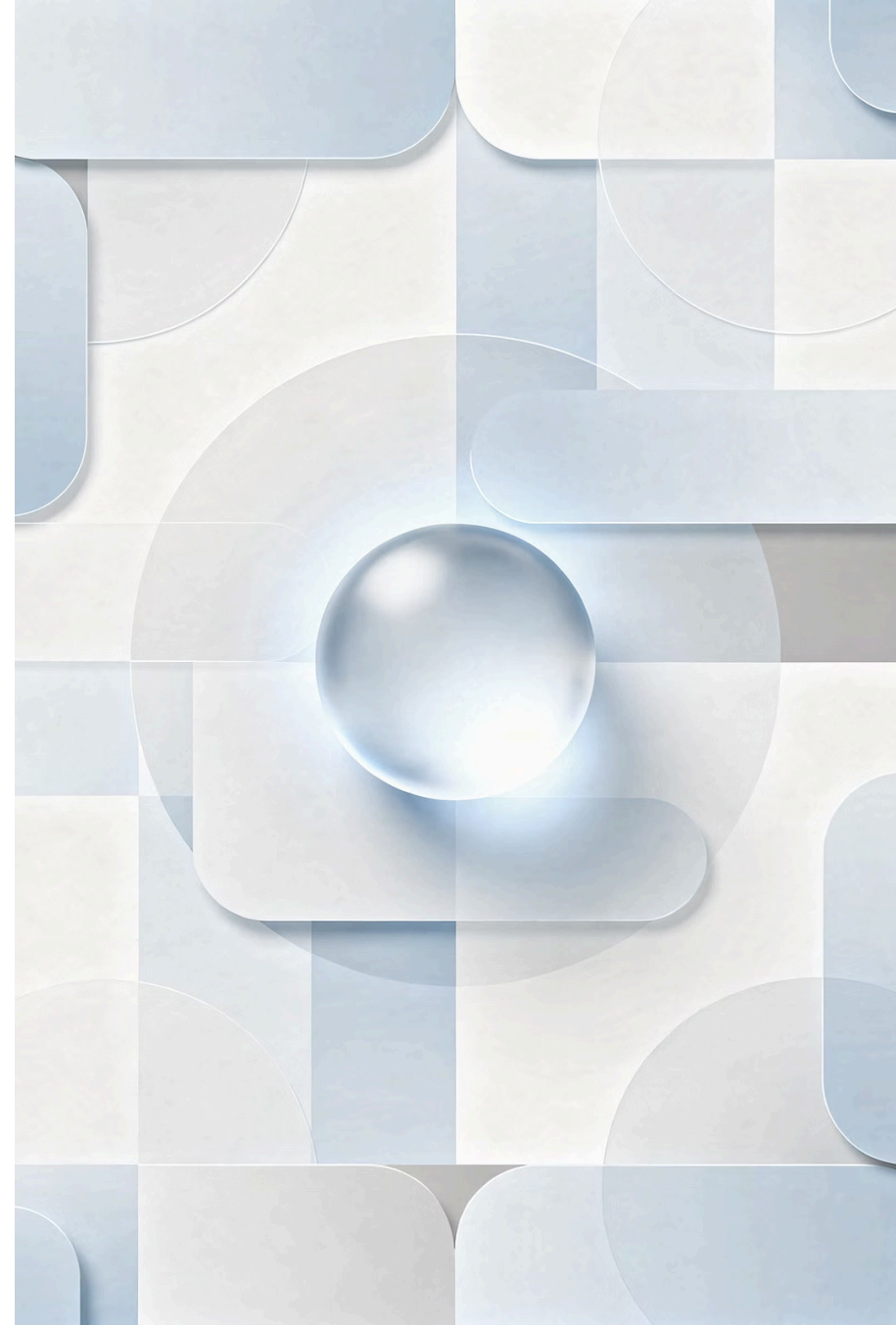


Patrón de Diseño Singleton

Instancia única — Un patrón de diseño creacional fundamental para controlar la creación de objetos y garantizar un punto de acceso global.



Agenda de la Presentación

01

Propósito

Qué es el Singleton y para qué sirve

03

Solución

Cómo se implementa el patrón

05

Aplicabilidad e Implementación

Cuándo y cómo usarlo

02

Problema

Los dos problemas que resuelve este patrón

04

Estructura y Pseudocódigo

Diagrama y ejemplo práctico

06

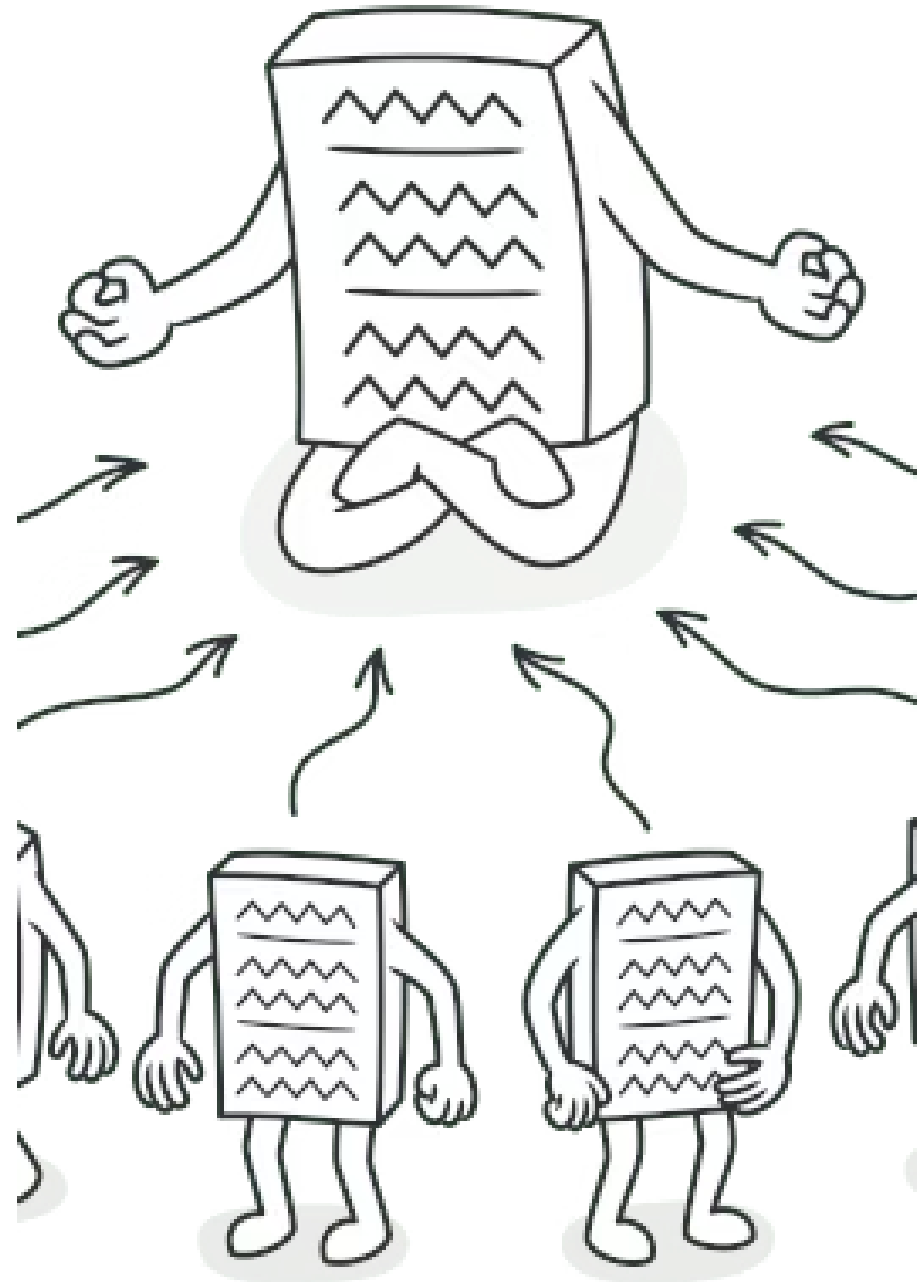
Pros, Contras y Relaciones

Ventajas, desventajas y vínculos con otros patrones

¿Qué es el Patrón Singleton?

Singleton es un **patrón de diseño creacional** que nos permite asegurarnos de que una clase tenga una **única instancia**, a la vez que proporciona un **punto de acceso global** a dicha instancia.

Es uno de los patrones más conocidos y utilizados en el desarrollo de software, especialmente cuando se necesita controlar el acceso a recursos compartidos.



Dos Garantías Fundamentales

Única Instancia

Garantiza que una clase tenga solo un objeto creado en toda la aplicación, sin importar cuántas veces se solicite.

Acceso Global

Proporciona un punto de acceso global a esa instancia, disponible desde cualquier parte del programa de forma controlada.

El Problema que Resuelve

El patrón Singleton resuelve **dos problemas al mismo tiempo**, lo cual vulnera el **Principio de responsabilidad única**. Estos dos problemas son la razón de ser del patrón y merecen un análisis detallado.

1

Garantizar una única instancia

Controlar cuántas instancias tiene una clase para proteger recursos compartidos.

2

Punto de acceso global

Acceder al objeto desde cualquier parte del programa de forma segura.

Problema 1: Única Instancia

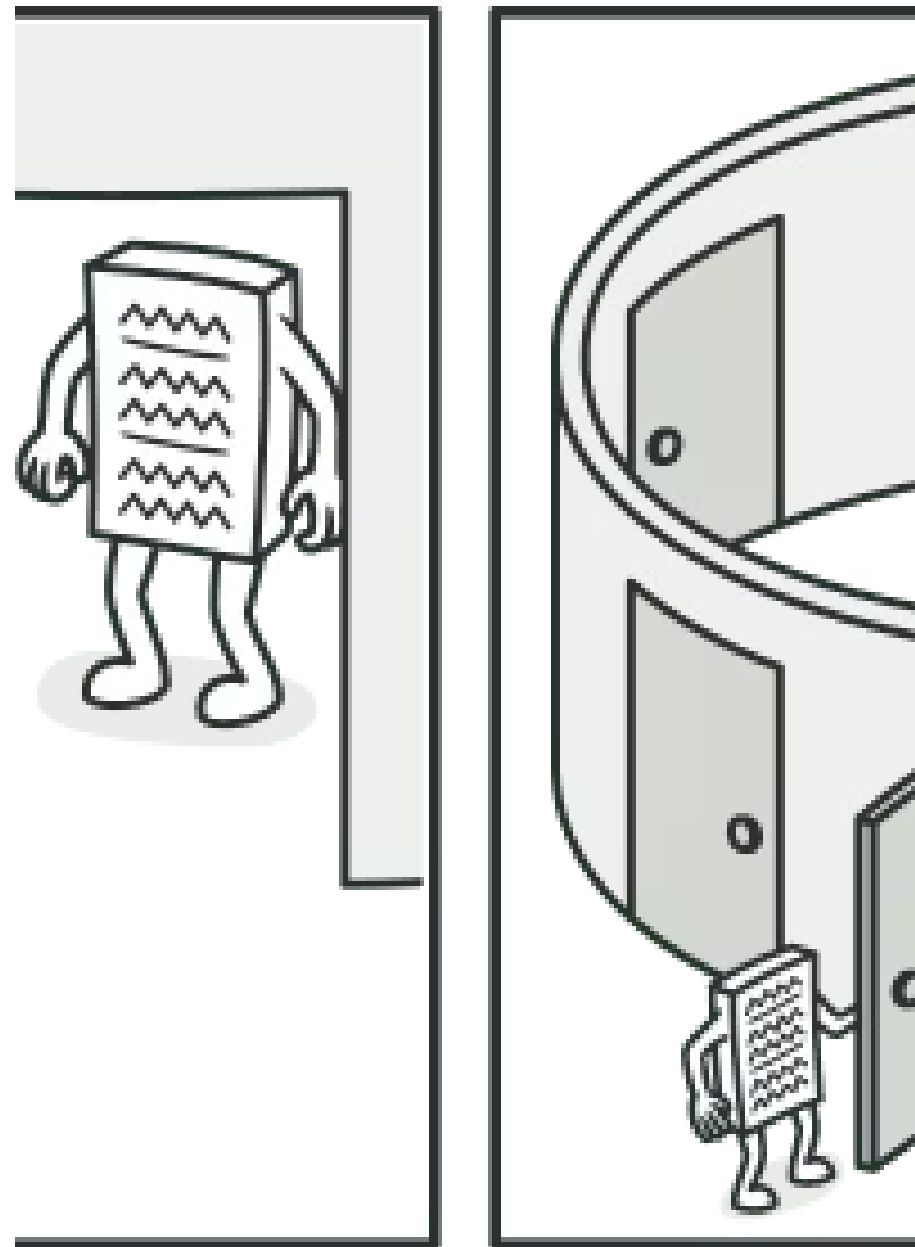
¿Por qué querría alguien controlar cuántas instancias tiene una clase? El motivo más habitual es **controlar el acceso a algún recurso compartido**, por ejemplo, una base de datos o un archivo.

❏ Funciona así: imagina que has creado un objeto y al cabo de un tiempo decides crear otro nuevo. En lugar de recibir un objeto nuevo, **obtendrás el que ya habías creado.**

Ten en cuenta que este comportamiento es **imposible de implementar con un constructor normal**, ya que una llamada al constructor siempre debe devolver un nuevo objeto por diseño.

Reutilización del Mismo Objeto

Cuando un cliente solicita una nueva instancia, el patrón Singleton devuelve la instancia ya existente en lugar de crear una nueva. Los clientes pueden ni siquiera darse cuenta de que trabajan con el mismo objeto todo el tiempo.



Problema 2: Acceso Global Seguro

¿Recuerdas esas **variables globales** que utilizaste para almacenar objetos esenciales? Aunque son muy útiles, también son **poco seguras**, ya que cualquier código podría sobrescribir el contenido de esas variables y descomponer la aplicación.

Variable Global

Cualquier código puede sobrescribir su contenido, rompiendo la aplicación.

Patrón Singleton

Permite acceder al objeto desde cualquier parte, pero evita que otro código sobreescriba esa instancia.

Código Centralizado y Organizado

Este problema tiene otra cara: **no queremos que el código que resuelve el primer problema se encuentre disperso por todo el programa**. Es mucho más conveniente tenerlo dentro de una clase, sobre todo si el resto del código ya depende de ella.

- ❏ Hoy en día el patrón Singleton se ha popularizado tanto que la gente suele llamar *singleton* a cualquier patrón, incluso si solo resuelve **uno** de los problemas antes mencionados.

La Solución del Singleton

Todas las implementaciones del patrón Singleton tienen estos **dos pasos en común**:

1

Constructor Privado

Hacer privado el constructor por defecto para evitar que otros objetos utilicen el operador `new` con la clase Singleton.

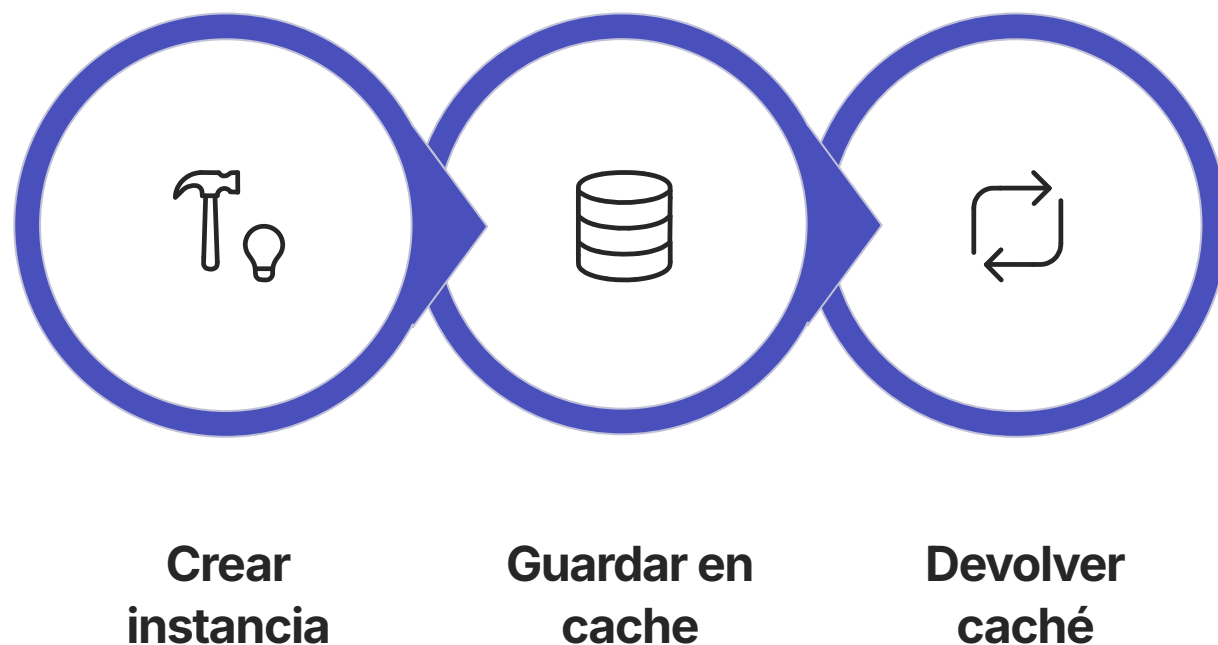
2

Método Estático

Crear un método de creación estático que actúe como constructor. Invoca al constructor privado y guarda el objeto en un campo estático.

¿Cómo Funciona el Método Estático?

Tras bambalinas, el método estático invoca al **constructor privado** para crear un objeto y lo guarda en un **campo estático**. Las siguientes llamadas a este método devuelven el **objeto almacenado en caché**.



Si tu código tiene acceso a la clase Singleton, podrá invocar su método estático. De esta manera, cada vez que se invoque este método, siempre se devolverá el mismo objeto.

Analogía en el Mundo Real

El **gobierno** es un ejemplo excelente del patrón Singleton. Un país solo puede tener **un gobierno oficial**. Independientemente de las identidades personales de los individuos que lo forman, el título "**Gobierno de X**" es un punto de acceso global que identifica al grupo de personas a cargo.



Singleton

- instance: Singleton

- Singleton()

+ getInstance(): Singleton

```
if (instance == null) {  
    // Nota: si estás creando  
    // que soporte el multithreading  
    // colocar un bloqueo de sincronización  
    instance = new Singleton()  
}  
return instance
```

CAPÍTULO 4 ESTRUCTURA

Estructura del Patrón

Método estático `obtenerInstancia`

La clase Singleton declara este método que devuelve la misma instancia de su propia clase.

Constructor oculto

El constructor del Singleton debe ocultarse del código cliente. La llamada al método `obtenerInstancia` debe ser la **única manera** de obtener el objeto.

Pseudocódigo: Conexión a Base de Datos

En este ejemplo, la clase de conexión de la base de datos actúa como Singleton. No tiene un constructor público, por lo que la única manera de obtener su objeto es invocando el método `obtenerInstancia`. Este método almacena en caché el primer objeto creado y lo devuelve en todas las llamadas siguientes.

Clase Database (Parte 1)

```
// La clase Base de datos define el método `obtenerInstancia`  
// que permite a los clientes acceder a la misma instancia de  
// una conexión de la base de datos a través del programa.  
class Database is  
    // El campo para almacenar la instancia singleton debe  
    // declararse estático.  
    private static field instance: Database  
  
    // El constructor del singleton siempre debe ser privado  
    // para evitar llamadas de construcción directas con el  
    // operador `new`.  
    private constructor Database() is  
        // Algún código de inicialización, como la propia  
        // conexión al servidor de una base de datos.  
        // ...
```

Clase Database (Parte 2)

```
// El método estático que controla el acceso a la instancia
// singleton.
```

```
public static method getInstance() is
```

```
if (Database.instance == null) then
```

```
  acquireThreadLock() and then
```

```
  // Garantiza que la instancia aún no se ha
```

```
  // inicializado por otro hilo mientras ésta ha
```

```
  // estado esperando el desbloqueo.
```

```
  if (Database.instance == null) then
```

```
    Database.instance = new Database()
```

```
  return Database.instance
```

```
// Por último, cualquier singleton debe definir cierta
```

```
// lógica de negocio que pueda ejecutarse en su instancia.
```

```
public method query(sql) is
```

```
  // Por ejemplo, todas las consultas a la base de datos
```

```
  // de una aplicación pasan por este método. Por lo
```

```
  // tanto, aquí puedes colocar lógica de regularización
```

```
  // (throttling) o de envío a la memoria caché.
```

```
  // ...
```


Clase Application

```
class Application is
  method main() is
    Database foo = Database.getInstance()
    foo.query("SELECT ...")
    // ...
    Database bar = Database.getInstance()
    bar.query("SELECT ...")
    // La variable `bar` contendrá el mismo objeto que la
    // variable `foo`.
```

- ❏ Observa cómo `foo` y `bar` apuntan al **mismo objeto**. Ambas variables obtienen la misma instancia a través del método `getInstance()`, demostrando el comportamiento central del Singleton.

¿Cuándo Utilizar el Singleton?



Instancia Única Requerida

Cuando una clase de tu programa tan solo deba tener una instancia disponible para todos los clientes; por ejemplo, un único objeto de base de datos compartido por distintas partes del programa.



Control de Variables Globales

Cuando necesites un control más estricto de las variables globales. El Singleton garantiza que haya una única instancia y que nada pueda sustituirla.

Comportamiento del Singleton

Creación controlada

El patrón Singleton **deshabilita el resto de las maneras de crear objetos** de una clase, excepto el método especial de creación. Este método crea un nuevo objeto, o bien devuelve uno existente si ya ha sido creado.

Flexibilidad

Siempre podrás ajustar esta limitación y permitir la creación de **cierto número de instancias** Singleton. La única parte del código que requiere cambios es el cuerpo del método `getInstance`.

- ❏ Al contrario que las variables globales, el patrón Singleton **garantiza** que haya una única instancia de una clase. A excepción de la propia clase Singleton, nada puede sustituir la instancia en caché.

Cómo Implementar el Singleton

1 Campo estático privado

Añade un campo estático privado a la clase para almacenar la instancia Singleton.

2 Método estático público

Declara un método de creación estático público para obtener la instancia Singleton.

3 Inicialización diferida

Implementa una inicialización diferida dentro del método estático. Debe crear un nuevo objeto en su primera llamada y colocarlo dentro del campo estático.

4 Constructor privado

Declara el constructor de clase como privado. El método estático seguirá siendo capaz de invocar al constructor, pero no los otros objetos.

5 Refactorizar el código cliente

Sustituye todas las llamadas directas al constructor por llamadas al método de creación estático.

Ventajas del Singleton



Instancia única garantizada

Puedes tener la certeza de que una clase tiene una única instancia en toda la aplicación.



Acceso global

Obtienes un punto de acceso global a dicha instancia desde cualquier parte del programa.



Inicialización diferida

El objeto Singleton solo se inicializa cuando se requiere por primera vez, optimizando recursos.

Desventajas del Singleton

Vulnera el Principio de Responsabilidad Única

El patrón resuelve dos problemas al mismo tiempo.

Puede enmascarar mal diseño

Por ejemplo, cuando los componentes del programa saben demasiado los unos sobre los otros.

Problemas con múltiples hilos

Requiere tratamiento especial en entornos multihilo para que varios hilos no creen un objeto Singleton varias veces.

Dificultad en pruebas unitarias

Muchos frameworks de prueba dependen de la herencia para crear mock objects. La clase Singleton es privada y en la mayoría de los lenguajes resulta imposible sobrescribir métodos estáticos.

Relaciones con Otros Patrones

El patrón Singleton se relaciona con varios otros patrones de diseño de formas interesantes:

Facade → Singleton

Una clase **Fachada** a menudo puede transformarse en un Singleton, ya que un único objeto fachada es suficiente en la mayoría de los casos.

Flyweight vs. Singleton

Flyweight podría asemejarse a Singleton si pudieras reducir todos los estados compartidos a un único objeto flyweight.

Abstract Factory, Builder, Prototype

Los patrones **Abstract Factory**, **Builder** y **Prototype** pueden todos ellos implementarse como Singletons.

Flyweight vs. Singleton: Diferencias Clave

Aunque Flyweight podría asemejarse a Singleton, existen **dos diferencias fundamentales** entre estos patrones:

Cantidad de instancias

Solo debe haber **una instancia Singleton**, mientras que una clase Flyweight puede tener **varias instancias** con distintos estados intrínsecos.

Mutabilidad

El objeto Singleton puede ser **mutable**. Los objetos Flyweight son **inmutables**.

Resumen y Conclusiones

El patrón Singleton garantiza una **única instancia** con **acceso global**, ideal para recursos compartidos como bases de datos. Aunque poderoso, debe usarse con precaución por sus implicaciones en responsabilidad única, multihilo y pruebas unitarias.

Constructor privado

Bloquea la creación directa de objetos

Método estático

Único punto de acceso controlado

Caché de instancia

Reutiliza siempre el mismo objeto

